

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente División
de Ciencias de la Ingeniería
Ing. Daniel Alberto González
Lenguajes Formales y de Programación
Sección: “A”



Manual Técnico

Analizador Léxico

Guillermo Emanuel Montejo Martínez
Carné: 202430624

Quetzaltenango, 17 de agosto de 2026

Introducción

El presente documento detalla la estructura interna y el funcionamiento de la primera fase del compilador para el lenguaje de programación **PromptZal**. Este lenguaje ha sido diseñado con el propósito de permitir a los desarrolladores redactar directivas y comandos estructurados que posteriormente se traducirán en prompts optimizados para herramientas de inteligencia artificial.

Objetivos

El objetivo central de este módulo es realizar el **Análisis Léxico** (tokenización) del código fuente escrito en archivos .pz. El sistema se encarga de leer el archivo de entrada, agrupar los caracteres en lexemas con significado y clasificarlos según la gramática del lenguaje, reportando de manera continua cualquier error léxico encontrado sin interrumpir el proceso de compilación.

Restricciones Técnicas Cumplidas:

- Desarrollo nativo en lenguaje Java.
- Implementación de un autómata finito determinista de forma manual, recorriendo la entrada carácter por carácter.
- No se utilizaron generadores automáticos (como JFlex o Lex) ni librerías de expresiones regulares.

Arquitectura del Sistema

El sistema ha sido diseñado bajo el paradigma de Programación Orientada a Objetos (POO), dividiendo el analizador en componentes modulares para asegurar el Principio de Responsabilidad Única.

1.1) Modelos de datos

- **TipoToken (Enum):** Define el catálogo completo de categorías léxicas reconocidas por el lenguaje (Directivas, Palabras Reservadas, Comandos de IA, Funciones, Conectores, Identificadores, Literales, Operadores y Delimitadores).
- **Token:** Entidad que almacena la información de un componente léxico válido, encapsulando su identificador correlativo, lexema, tipo (TipoToken), y la posición exacta (fila y columna) donde inicia.
- **ErrorLexico:** Entidad que registra los caracteres no reconocidos o estructuras incompletas (como cadenas sin cerrar), almacenando la descripción del fallo y su ubicación para fines de depuración.

1.2) Motor de Análisis

Es la clase núcleo del sistema. Recibe el código fuente convertido en un arreglo de caracteres (`char[]`) e implementa un ciclo de evaluación secuencial. Mediante un sistema de delegación, identifica el tipo de carácter actual y transfiere el control a submétodos especializados (ej. `manejarPalabra()`, `manejarCadena()`, `manejarNumero()`) que se encargan de ensamblar el lexema completo empleando técnicas de anticipación (*lookahead*).

1.3) Capa de Presentación y Reportes

Una vez finalizado el análisis, este módulo procesa las listas resultantes en memoria y construye dinámicamente dos documentos en formato HTML:

- **Reporte de Tokens:** Despliega una tabla detallada con todos los lexemas reconocidos exitosamente.
- **Reporte de Errores:** Expone una tabla con las fallas léxicas; en caso de un archivo completamente válido, inyecta un mensaje confirmando la ausencia de errores.

Listado Oficial de Tokens

El autómata finito determinista (AFD) fue diseñado para reconocer y clasificar de manera precisa las distintas familias de tokens definidas en la gramática de PromptZal. A continuación se detalla el catálogo de tokens soportado por el analizador:

Categoría	Descripción	Lexemas Reconocidos	Tipo de Token
Directivas	Metadatos que definen propiedades del prompt a generar. Inician con el símbolo @.	@modelo, @rol, @formato	DIRECTIVA_MODELO, DIRECTIVA_ROL, etc.
Palabras Reservadas	Definen la estructura principal del programa y la ejecución de los agentes.	Agente, contexto, variable, EJECUTAR, EXPORTAR.	RESERVADA_AGENTE, RESERVADA_CONTEXTO, etc.
Comando de IA	Acciones específicas que se solicitarán a la inteligencia artificial.	PREGUNTAR, GENERAR, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER	COMANDO_PREGUNTA, COMANDO_GENERAR, etc.
Funciones	Instrucciones predefinidas para la carga de datos externos.	CARGAR	FUNCION_CARGAR
Conectores	Unen los comandos con sus respectivos argumentos y variables de salida.	SOBRE, DESDE, EN, COMO, ->	CONECTOR_SOBRE, CONECTOR_FLECHA, etc.
Literales	Valores estáticos ingresados “...” (cadenas), Enteros por el programador. Incluye texto y números.	(ej. 10), Decimales(ej. 0.7)	LITERAL_CADENA, LITERAL_ENTERO, LITERAL_DECIMAL
Identificadores	Nombres definidos por el usuario. Deben iniciar con letra o guion bajo.	ej. traductor, base_datos	IDENTIFICADOR
Operadores	Símbolos para asignación de variables y concatenación de cadenas.	Signo igual y signo suma	OPERADOR_ASIGNACION, OPERADOR_CONCATENACION
Delimitadores	Símbolos para agrupar bloques lógicos, funciones o listas.	{, }, (,), ,	LLAVE_ABRE, PARENTESIS_CIERRE, COMA, etc.

Motor de Análisis y Recorrido de la Entrada

El cumplimiento del requisito de tokenización manual (sin herramientas automatizadas) se logró implementando un algoritmo de lectura secuencial dentro de la clase AnalizadorLexico.

1.1) Control de punteros

El método principal analizar(char[] entrada) utiliza un ciclo while controlado por un puntero principal i que recorre el arreglo desde la posición 0 hasta la longitud total del archivo. De manera paralela, el sistema administra dos variables contadoras (fila y columna) que se actualizan dinámicamente:

- La variable columna incrementa con cada lectura de carácter.
- Al detectar un salto de línea (\n), la variable fila incrementa en 1 y la columna se reinicia a 1.

1.2) Algoritmo de Identificación y Anticipación (Lookahead)

Para clasificar los componentes, el programa evalúa el carácter actual mediante métodos booleanos (ej. esLetraOGuionBajo(), esNumero()). Dependiendo del resultado, el flujo se desvía a métodos de ensamblaje (lookahead) que "miran hacia adelante" para construir el lexema completo:

- **Identificadores y Palabras Reservadas:** El método manejarPalabra() concatena caracteres mientras sean letras, dígitos o guiones bajos. Al encontrar un carácter distinto, el ciclo se detiene. El lexema ensamblado pasa por un evaluador switch (clasificarPalabra()) que determina si es una palabra reservada del lenguaje o, por defecto, un identificador general.
- **Números:** El método manejarNumero() agrupa dígitos continuos. Si detecta un punto ., activa una bandera booleana. Si encuentra un segundo punto, el ensamblaje se corta para evitar errores lógicos. Al finalizar, la bandera booleana determina si el número se guarda como LITERAL_ENTERO o LITERAL_DECIMAL.
- **Conectores Compuestos:** Al detectar un guion -, el método manejarOperadores() evalúa inmediatamente si el carácter i + 1 es un >. Si es así, se ensambla el conector flecha -> y se avanzan dos posiciones; de lo contrario, se reporta un error léxico.

Manejo y Recuperación de Errores Léxicos

El compilador PromptZal cuenta con un mecanismo de resiliencia ante fallas léxicas. Siguiendo estrictamente las especificaciones del proyecto, el análisis nunca se detiene ante un error.

1.1) Tipos de Errores Controlados

- **Caracteres no reconocidos:** Cualquier símbolo que no pertenezca al alfabeto de PromptZal (ej. \$, %, ?).
- **Cadenas sin cerrar:** Un literal de texto que comienza con " pero alcanza un salto de línea (`\n`) o el final del archivo sin encontrar la comilla de cierre.
- **Comentarios sin cerrar:** Un comentario de bloque que inicia con `/*` pero nunca se cierra con `*/`.
- **Estructuras incompletas:** Directivas mal escritas (ej. `@modelo_falso`) o símbolos sueltos (ej. un `/` que no forma parte de un comentario).

1.2) Estrategia de Recuperación

Cuando el autómata detecta una anomalía (cayendo en el bloque `else` del ciclo principal o fallando una validación interna), se invoca el método de contingencia. Este método instancia un objeto de tipo `ErrorLexico` guardando el carácter problemático, su descripción y su ubicación exacta. Inmediatamente después, el puntero `i` y la columna avanzan una posición, permitiendo que el ciclo `while` retome la evaluación normal en el siguiente carácter, garantizando así la recolección exhaustiva de todos los errores presentes en el archivo.

Diagrama UML

